

SWEMLS: Coursework 2: Inference Design

[Live version of this document in Google Docs](#)

Background

The clinical leadership of South Riverside Hospital are pleased with the progress they've seen on our predictive model for acute kidney injury. They would like us to start thinking about how to deploy it within the hospital. To develop our model, we used a static export of patient data from the hospital's research data warehouse, represented as a CSV file.

In the hospital environment, we'll be listening to a real-time stream of messages sent from hospital systems detailing patients entering the hospital, leaving the hospital, and the results of their blood tests. If we predict a patient has developed an acute kidney injury, we should alert their clinical response team by making a request to their pager management system with the patient's details.

The hospital's systems will send us messages over a TCP/IP socket in the HL7 message format, wrapped in the simple MLLP protocol. Both HL7 and MLLP are introduced below. We'll integrate with their pager system via HTTP. New blood test results will be received as HL7 messages. Historical blood test results for patients will be provided as a CSV export, similar to the training data used to build our model, although including unique identifiers for patients. This initial CSV file will not be updated. Once we've been sent a blood test result as a HL7 message, we won't be told about it again.

Note: HL7 and MLLP are the real protocols used in hospitals, although we'll only be using a tiny subset of what can be represented in HL7 messages. The protocols are simple enough for you to write code to support from first principles, but open source libraries also exist for working with them (for example, in [Python](#)).

Task

For this coursework, your task will be to form groups, and within those groups write a high-level design document for a system that will use a model from the first coursework to detect kidney injury in the hospital environment. You'll build the system you design in future courseworks.

Design documents

Design documents are relatively informal descriptions of why a piece of software is being built, and the trade-offs made by its design. They provide higher-level context than the code itself, its comments and change descriptions, which generally describe how a part of the software works. They are as much a tool to help the authors think through the software they're about to build in a structured way, as they are a mechanism to communicate that thinking to others.

As informal documents, there's generally no specific template to follow, and authors generally adapt their document structure to the problem. We've provided a [template for you to follow](#), if you'd like, but you may choose another. Examples of good design documents from open source projects include:

- [Python Enhancement Proposals](#)
- [Tensorflow RFCs](#)
- [IETF RFCs](#), for example [HTTP/1.1](#)
- [Spring 83](#), by [Robin Sloan](#), author of the novel [Sourdough](#)

Marking

The marks for this coursework will be broken down as follows:

- 70% communication of design
- 30% technical quality of design

Hospital system architecture crash course

The hospital makes use of many different systems, but we'll only need to worry about two - the patient administration system, or PAS, and the laboratory information management system, or LIMS.

The PAS keeps track of who's in the hospital, their demographics, their physical location, and many other attributes. You can consider a patient to be the primary type of object the PAS system cares about, and they're uniquely identified by a medical record number, or MRN. MRNs are unique to a patient, and will match those in historical data given to the system as a CSV file. They do not change if patients are readmitted to hospital.

The LIMS system keeps track of orders for diagnostic tests, and the results of those tests. You can consider orders and results to be the primary types of object the LIMS system cares about. Orders and results are tied back to patients by their MRN.

Both the PAS and the LIMS systems integrate via the HL7 protocol, although they send different messages. Our system will see a single stream of messages, but downstream, those messages are generated by two different, independent, systems.

HL7v2 messages

HL7v2 is a worldwide standard for integrating healthcare systems through message passing. The standard specifies how to construct and understand messages, but not how to send and receive them. Messages are 7-bit ASCII, and consist of one or more segments. Each segment is a line of text, ending with a carriage return, ie (\r or 0x0d). Segments have a 3 letter code identifying their type (eg MSH for message header), and a series of fields separated by pipe characters (ie | or 0x7c). Fields themselves can be broken down recursively into sub-fields,

normally using a caret character (ie ^ or 0x5e) for the first level, although this can be changed in the header.

In documents and API, the fields are referred to by their segment name and index within that segment, for example, [PID.2](#) is "Patient external identifier". The first data field is field 1 - you can consider the segment type, eg PID, to be field 0.

While the HL7 specification is long, we only care about a small number of message types. We give examples of those below - note that we have used newlines between segments for readability. The HL7 specification requires carriage returns only (ie \r or 0x0d).

An [archive of the messages below can be found here](#).

PAS system messages

Admitting a patient to hospital, represented by message type [ADT A01](#):

```
MSH|^~\&|SIMULATION|SOUTH RIVERSIDE|||202401201630||ADT^A01|||2.5
PID|1||478237423||ELIZABETH HOLMES||19840203|F The MRN, name, date of birth and sex of the patient
NK1|1|SUNNY BALWANI|PARTNER The next of kin of the patient
```

Discharging a patient from hospital, represented by message type

```
MSH|^~\&|SIMULATION|SOUTH RIVERSIDE|||202401201630||ADT^A03|||2.5
PID|1||478237423 The MRN of the patient being discharged
```

LIMS system messages

Receiving the result of a creatinine blood test result, represented by message type [ORU R01](#):

```
MSH|^~\&|SIMULATION|SOUTH RIVERSIDE|||202401201630||ORU^R01|||2.5
PID|1||478237423 The MRN of the patient the test relates to
OBR|1|||||202401202243 The time at which the blood test took place
OBX|1|SN|CREATININE||103.4 The result type (CREATININE), and the associated value (103.4)
```

Fields used in our hospital system

The following fields are guaranteed to be filled in messages from our hospital system. Others may, or may not, be filled.

[MSH.7](#) - Message date and time, in all messages

[MSH.9](#) - Type of message, eg ADT^A01 for an admit message

[PID.3](#) - Patient internal identifier, in all PID segments

[PID.5](#) - Patient name, in PAS messages only

[PID.7](#) - Patient date of birth, in PAS messages only

[PID.8](#) - Patient sex, in PAS messages only

[OBR.7](#) - The time at which a blood test took place in LIMS messages only

[OBX.3](#) - The type of blood test (CREATININE for creatinine), in LIMS messages only

[OBX.5](#) - The observed value of the blood test result, in LIMS messages only

Minimal lower layer protocol (MLLP)

Minimal lower layer protocol (MLLP) is a simple protocol for transmitting HL7 messages over a TCP/IP connection. The HL7 message standard doesn't include a mechanism for separating multiple messages, which would be required for sending messages across a single stream. MLLP adds this by surrounding messages with a start of block marker (defined as 0x0b) and an end of block marker (defined as 0x1c), and following it with a carriage return (ie 0x0d). It's agnostic to the content of the HL7 message itself.

Pager system

To page the hospital's clinical response team, make a HTTP post request to /page, with the MRN of the relevant patient as the body of the post, and the timestamp for the blood test result triggering the alert. Without a timestamp, we assume the time of the last test result sent for that patient. The timestamp uses the standard HL7 format.

For example, to issue a page for the patient shown in the example messages above, the request would look like:

```
POST /page HTTP/1.0
Content-type: text/plain
```

```
478237423,202401202243
```

Throughput and latency expectations

The hospital has around 800 beds. You can expect around 20 admissions (and therefore discharges) per day, and around 100 creatinine blood test results. The time between receiving a HL7 message with a creatinine result, and paging the clinical response team should the result indicate acute kidney injury, should be under 3 seconds.

You can assume that later in the module, your design will be deployed on a single-core virtual machine with 1Gb of RAM, managed by Kubernetes. It's unlikely that the technical details of Kubernetes will influence your design at this stage, but you should assume that your service can be stopped and restarted arbitrarily, and without notification, by the underlying cloud infrastructure to deal with operational issues, such as rolling machine upgrades.